

Strukture podataka i algoritmi

Python (2. deo)

dr Danijel Čabarkapa

Akademija strukovnih studija Šabac

Odsek za medicinske i poslovno-tehnološke studije

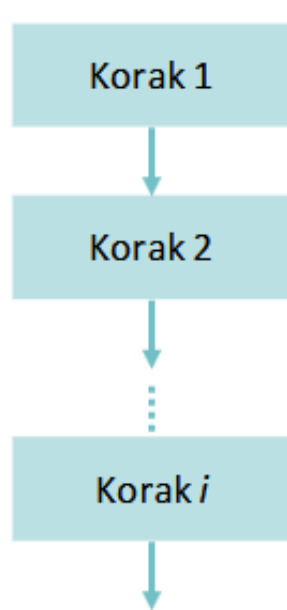
Sadržaj:

1. Kontrolne strukture
2. Python funkcije
3. Python objektno-orijentisano programiranje (klase)

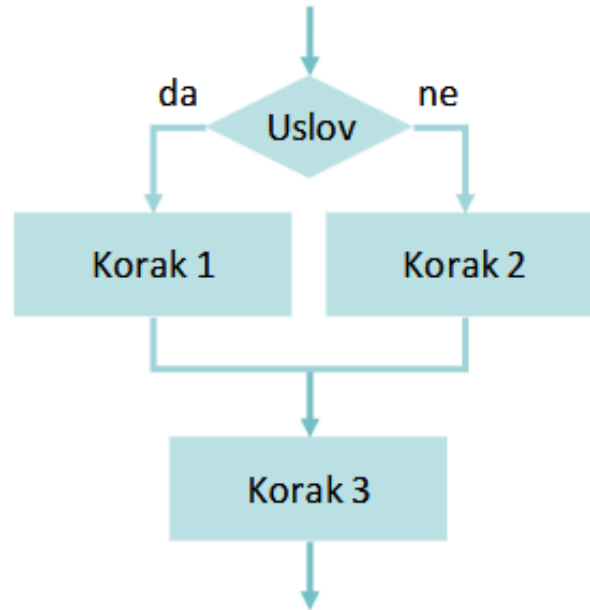
1. Kontrolne strukture

Kontrolne strukture

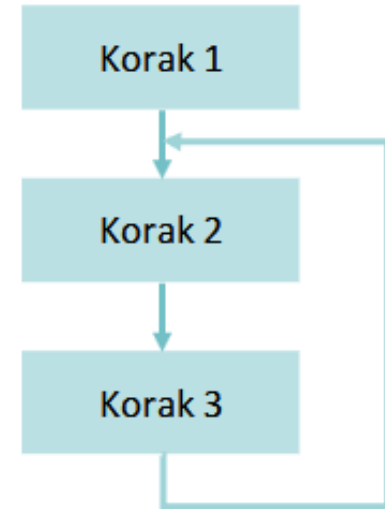
- Za realizovanje bilo kojeg algoritma dovoljna su tri načina upravljanja redosledom izvršavanja pojedinih koraka algoritma



Sekvencijalno
izvršavanje



Uslovno
izvršavanje (grananje,
selekcije)



Iterativno
izvršavanje (petlje,
ciklusi)

Kontrolne strukture

- Kao i drugi programski jezici, Python koristi kontrolne strukture za kontrolu toka programa i obezbeđivanje da određeni delovi programa budu izvršeni ako su ispunjeni određeni uslovi
- Python koristi dve vrste kontrolnih struktura:
 - a) Uslovna grananja (selekcije) - omogućava se izbor jednog od nekoliko ishoda
 - b) Petlje (ciklusi) - omogućavaju ponavljanje zadatih instrukcija određeni broj puta (brojačke petlje) sve dok je određeni uslov ispunjen
- Uslovi se uglavnom definišu korišćenjem logičkih tipova podataka
- Za predstavljanje rezultata logičkih i relacionih operatora koriste se samo dve vrednosti:
 - Logička istina – **TRUE**
 - Logička neistina – **FALSE**

Kontrolne strukture

- Logičkom neistinom se u Pythonu smatraju:
 - Vrednosti False i None
 - Vrednost 0 bilo kog numeričkog tipa: 0, 0.0, 0j
 - Prazne sekvence i kolekcije
 - Postoji funkcija `bool()` za eksplicitnu konverziju tipa
 - Sve druge vrednosti se smatraju logičkom istinom True
- Python podržava grananja prema logičkom uslovu, odnosno nizu povezanih logičkih uslova
- Python podržava ponavljanje unapred određen broj puta i ponavljanje prema logičkom uslovu koji se proverava pre početka ponavljanja
- Blokovi naredbi u jeziku Python označeni su uvlačenjem (indentacijom), na šta treba obratiti pažnju

Logički operatori

Primer: $a=10$, $b=20$

Operator	Opis	Primeri
and - logičko I x and y	Ako je x neistinito, rezultat je x , u suprotnom y	$(a \text{ and } b) \rightarrow 20$ (True)
or - logičko ILI x or y	Ako je x neistinito, rezultat je y , u suprotnom x	$(a \text{ or } b) \rightarrow 10$ (True)
not - logička negacija not x	Negira logičko stanje svog operanda x	$\text{not } (a \text{ and } b) \rightarrow \text{False}$



I

ILI

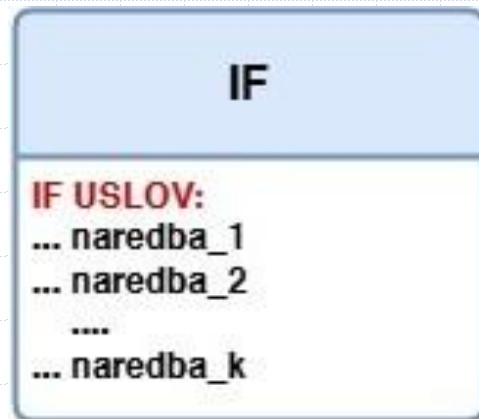
x	y	x and y	x or y
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Relacioni operatori

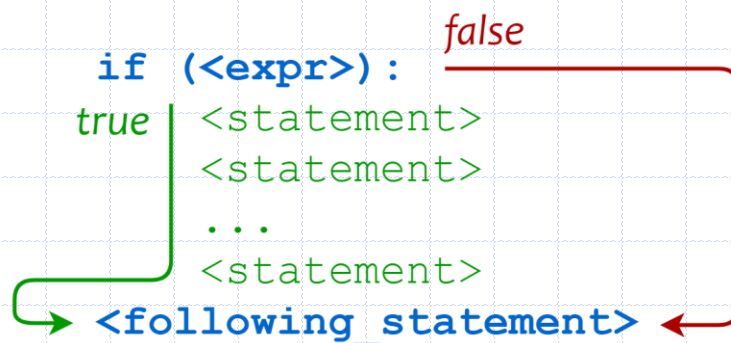
Operator	Opis	Primeri
==	Ako su vrednosti oba operanda jednake, rezultat je tačan	$(a == b) \rightarrow \text{False}$
!=	Ako su vrednosti oba operanda jednake, rezultat je netačan	$(a != b) \rightarrow \text{True}$
>	Ako je vrednost levog operanda veća od vrednosti desnog operanda, rezultat je tačan	$(a > b) \rightarrow \text{False}$
<	Ako je vrednost levog operanda manja od vrednosti desnog operanda, rezultat je tačan	$(a < b) \rightarrow \text{True}$
>=	Ako je vrednost levog operanda veća ili jednaka od vrednosti desnog operanda, rezultat je tačan	$(a >= b) \rightarrow \text{False}$
<=	Ako je vrednost levog operanda manja ili jednaka od vrednosti desnog operanda, rezultat je tačan	$(a <= b) \rightarrow \text{True}$

Uslovna grananja (if, else, elif)

- Neke naredbe se izvršavaju samo ako je neki uslov ispunjen
- Da bi se opisalo uslovno izvršavanje naredbi koristi se naredba **IF**
- Zgrade u uslovu nisu OBAVEZNE (koristi se uvlačenje kôda)
- Dvotačka je obavezna nakon navođenja uslova
- Naredba uslova će vratiti tačno/netačno, ako je tačan uslov koji se zadaje izvršiće se naredba_1, itd. redom...



Uslovna grananja (IF)



- Osnovni IF iskaz:
 - ▶ Izraz uslov (**expr**) mora biti izraz logičkog tipa i mora se evaluirati u vrednost **True** ili **False**
 - ▶ Formira se korišćenjem relacionih i logičkih operatora, ili kao rezultat poziva funkcije
 - ▶ Simbol dvotačka (**:**) zadaje početak novog bloka koda i obavezno se navodi
 - ▶ Blok koda sadrži proizvoljan broj naredbi, ako se sastoji od samo jedne naredbe ona se može navesti u produžetku iza znaka (**:**)
 - ▶ Ako ima više naredbi u bloku, svaka naredba mora biti u novom redu, podjednako uvučena u odnosu na **if** da bi se smatrala delom istog bloka

Uslovna grananja (if)

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>> x=0
>>> y=5
>>> if x<y:
...     print('Yes')
...
Yes
>>> if y<x:
...     print('Yes')
...
>>> if x:
...     print('Yes')
...
>>> if y:
...     print('Yes')
...
Yes
>>> if 'eo' in 'Beograd':
...     print('Yes')
...
Yes
>>> if 'eo' in ['bar', 'baz', 'qux']:
...     print('Iskaz je tacan')
...     print('Izvravanje u bloku')
...     print('...')
...     print('OK!')
... print('Uslov nije ispunjen')
SyntaxError: invalid syntax
>>> print('Uslov nije ispunjen')
Uslov nije ispunjen
>>>
```

greška uvlačenja
reda!!!

Uslovna grananja (IF)

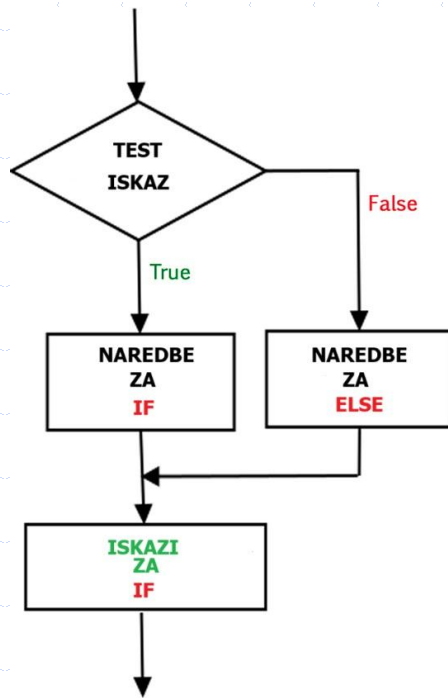
```
>>> a=0
>>> if a:
...     print(a)
...
>>>
>>> a=0
>>> if a==0:
...     print(a)
...
0
>>> |
```

- Za kontrolu toka Python razlikuje 0 i null sa jedne strane i nešto različito od 0 ili null sa druge strane

```
var1 = 100
if var1:
    print ("1 - Got a true expression value")
    print (var1)
var2 = 0
if var2:
    print ("2 - Got a true expression value")
    print (var2)
print ("Good bye!")
```

```
1 - Got a true expression value
100
Good bye!
```

Uslovna grananja (IF-ELSE)



```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help

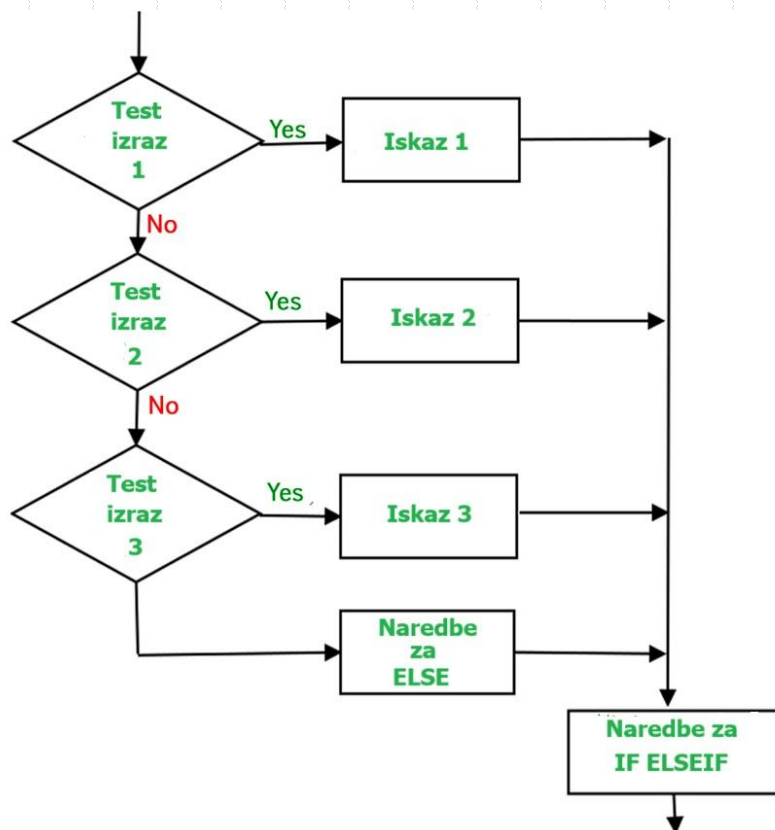
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MS
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>> a=int(input("Unesi stranicu a:"))
Unesi stranicu a:5
>>> b=int(input("Unesi stranicu b:"))
Unesi stranicu b:8
>>> if a==b:
...     p=a**2
...     print ("Povrsina kvadrata je:",p)
... else:
...     p=a*b
...     print ("Povrsina pravougaonika je:",p)
...
Povrsina pravougaonika je: 40
>>>
```

```
var1 = 0
if var1:
    print ("1 - true expression value")
    print (var1)
else:
    print ("2 - false expression value")
    print (var1)
```

2 - false expression value
0

Uslovna grananja (IF-ELIF-ELSE)

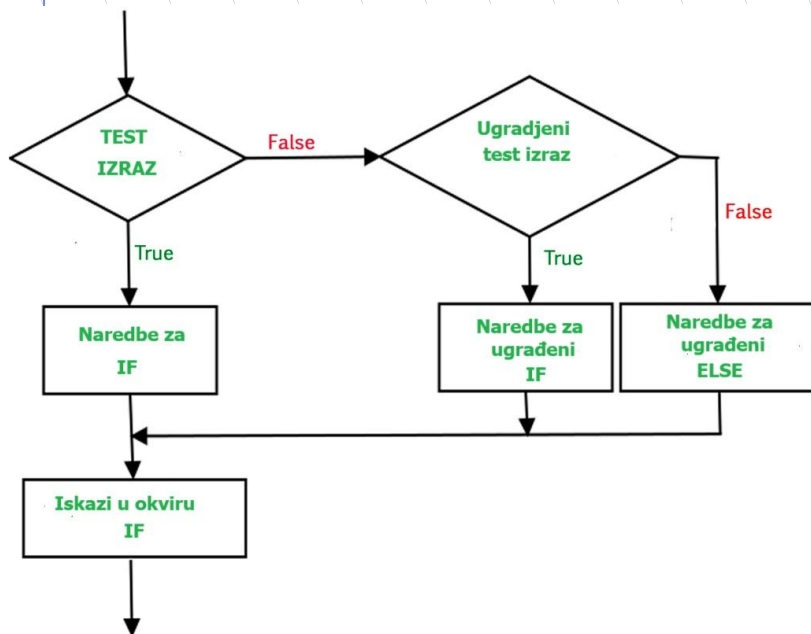
Ako je potrebno da se testira više od 2 moguće situacije, koristi se sintaksa `if-elif-else`; U ovoj formi Python izvršava samo jedan blok, pokreće svaki uslovni test po redu, sve dok jedan ne bude uspešan; Kada je test uspešan, izvršen je kod koji prati taj test i Python preskače ostale testove



```
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2024, AMD64) on win32
Enter "help" below or click "Help" above for more
>>> #Primer if-elif iskaza
>>> slovo="A"
>>> if slovo=="B":
...     print("Slovo je B")
... elif slovo=="C":
...     print("Slovo je C")
... elif slovo=="A":
...     print("Slovo je A")
... else:
...     print("Slovo nije A, B, ni C")
...
Slovo je A
>>>
```

Višestruki ugnežđeni IF uslov

- Ugnežđeni **if** iskazi omogućavaju da se implementiraju složeni logički tokovi
- Unutrašnji **if** iskazi se procenjuju samo ako je spoljašnji uslov istinit
- Svaki ugnježđeni **if** (ili **elif**, **else**) blok mora biti uvučen u odnosu na svoj nadređeni blok i obično se koristi 4 razmaka po nivou uvlačenja



```
kupac_je_clan = True
ukupna_potrosnja = 150
broj_artikala = 3

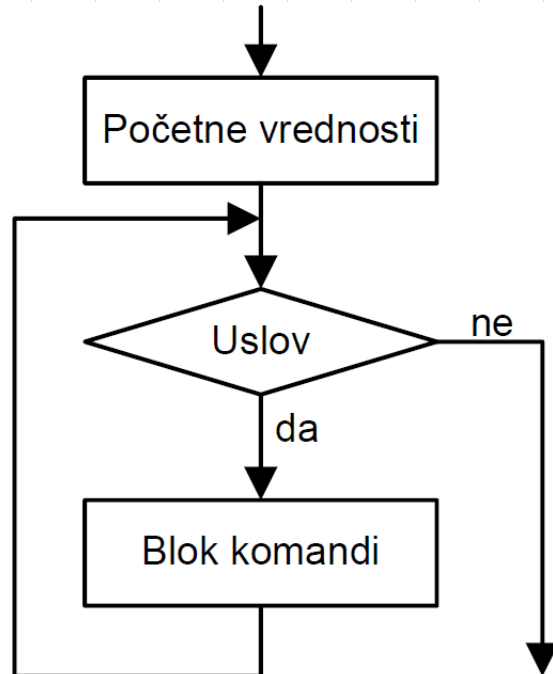
if kupac_je_clan:
    print("Kupac je član.")
    if ukupna_potrosnja > 100:
        print("Potrošnja prelazi 100.")
        if broj_artikala >= 2:
            print("Broj artikala je 2 ili više. Ostvarujete dodatni popust!")
        else:
            print("Broj artikala je manji od 2.")
    else:
        print("Potrošnja ne prelazi 100.")
else:
    print("Kupac nije član.")
```

Python ponavljanja i iteracije

- Petlje (ciklusi, iteracije) se koriste kada god je neku obradu potrebno izvršiti veći broj puta
- Najčešće je potrebno određene linije kôda ponoviti određeni zadatak broj puta kako bi se određeni uslov ispunio
- Python podržava dva tipa petlji:
 - a) **WHILE** (za iteriranje do zadovoljenja nekog opšteg uslova)
 - b) **FOR** (za iteriranje kroz kolekcije podataka ili u određenom opsegu) i u osnovi je to brojački ciklus

Petlja WHILE

- Petljom WHILE ponavlja se blok naredbi dok god je zadati logički uslov ispunjen tj. uslov mora da bude istinit pre izvršavanja petlje da bi se blok naredbi izvršio
- Provera uslova se vrši pre izvršavanja tela petlje
- To je ciklus sa izlaskom na vrhu koji se izvršava 0 ili više puta
- Telo petlje se može sastojati od jedne ili više naredbi, podjednako uvučenih u odnosu na ključnu reč **while**



while (uslov): !!!
naredba 1
naredba 2

Važno je osigurati da se uslov na kraju promeni u False kako bi se izbegla beskonačna petlja!!!

Petlja WHILE

- Poređenje naredbi IF i WHILE

```
br = 0
if br < 5:
    print('Zdravo!')
    br = br + 1
```

Zdravo!

```
br = 0
while br < 5:
    print('Zdravo!')
    br = br + 1
```

Zdravo!
Zdravo!
Zdravo!
Zdravo!
Zdravo!

```
>>> n=10
>>> while n>0:
...     print(n)
...     n=n-1
...
10
9
8
7
6
5
4
3
2
1
n=5
>>> while n>0:
...     print(n)
...     #Izostavljen uslov za prekid petlje
...
5
5
5
5
5
5
```

Petlja WHILE se neprekidno izvršava
jer nije postavljen izraz za prekid

Petlja WHILE

Primer: Odrediti koren svih celih brojeva od 1 do n ($n > 1$) i prikazati ih tabelarno

```
import math
```

```
n=int(input())
```

```
br=1
```

```
while br <= n:
```

```
    print(br, '    ', math.sqrt(br))
```

```
    br=br+1
```

5

1

1.0

2

1.4142135623730951

3

1.7320508075688772

4

2.0

5

2.23606797749979

Primer 3: Petlja sa uslovom i 'break' iskazom

```
i = 0
```

```
while True: # Petlja koja naizgled radi beskonačno jer je uslov uvek True.
```

```
    print(f"Trenutna vrednost i: {i}")
```

```
    i = i + 1
```

```
    if i > 5:
```

```
        print("Dostigli smo granicu, izlazimo iz petlje.")
```

```
        break # 'break' iskaz se koristi za momentalni izlazak iz najbliže WHILE
```

```
print("Petlja sa break-om završena.")
```

Naredba BREAK se koristi za trenutni izlaz iz najbliže WHILE petlje

Petlja WHILE

Primer petlje while:

```
count = 0
while (count < 9):
    print ('The count is:', count)
    count = count + 1
```

Primer beskonačne while petlje:

```
var = 1
while var == 1 : # infinite loop
    num = int(input("Enter a int number :"))
    print ("You entered: ", num)
```

Primer kombinacije while i else:

```
count = 0
while count < 5:
    print (count, " is less than 5")
    count = count + 1
else: print (count, " is not less than 5")
```

Petlja FOR

- Petljom FOR ponavlja se blok naredbi određeni broj puta
- Ponavljanje se vrši u određenom, zadatom opsegu za svaki element neke kolekcije podataka
- Nema logičkog uslova, već se petlji zadaje neki objekat koji omogućava iterativnu proceduru
- Za brojačke petlje (kada je broj ponavljanja poznat) pogodnije je koristiti **FOR**
- Sintaksa:

```
for brojač in opseg :  
    telo_petlje
```

- Promenljiva petlje **brojač** redom uzima vrednosti iz **opsega**
- Za svaku vrednost iz opsega se obavlja jedna iteracija

Petlja FOR

- Opseg ili skup vrednosti može biti eksplicitno zadan nabrojanjem pojedinačnih vrednosti ili pomoću funkcije `range()`

`range(stop)`

`range(start, stop, [step])`

- `range()` generiše sekvencu u opsegu `[start, stop)`
- Ukoliko se ne zada start, podrazumeva se 0, donja granica se uključuje, gornja ne
- Polje `step` definiše korak koji mora biti različit od 0 i može biti negativan (omogućava iteraciju unazad)

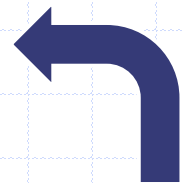
```
range(1, 10, 1)  skup vrednosti 1, 2, 3, 4, 5, 6, 7, 8, 9
range(20, 1, -2) skup vrednosti 20, 18, 16, 14, 12, 10, 8, 6, 4, 2
range(10)        skup vrednosti 0, 1, ..., 9
range(1, 11)     skup vrednosti 1, 2, ..., 10
range(0, 10, 3)  skup vrednosti 0, 3, 6, 9
range(0, -10, -1) skup vrednosti 0, 1, 2, ..., -9
range(0)         prazan skup vrednosti
range(1, 0)      prazan skup vrednosti
```

Petlja FOR

- Primer: Ispis sume svih parnih brojeva u zadatom opsegu od 1 do n

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5 AMD64) on win32
Enter "help" below or click "Help"
>>> n=int(input("Unesi n: "))
Unesi n: 20
>>> suma=0
>>> for i in range(2,n+1,2):
SyntaxError: expected ':'
>>> for i in range(2,n+1,2):
...     suma+=i
...
...
>>> print(suma)
110
>>>
```

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5 AMD64) on win32
Enter "help" below or click "Help"
>>> n=int(input("Unesi n: "))
Unesi n: 20
>>> suma=0
>>> for i in range(1, n+1):
...     suma+=i if i%2==0 else 0
...
...
>>> print(suma)
110
>>>
```



Sa proverom
parnosti

```
for i in range(25,4,-5):
    print (i)
```

25
20
15
10
5

```
for i in range(5,25,5):
    print (i)
```

5
10
15
20

Komande BREAK i CONTINUE

- Ako postoji potreba da petlja ranije završi svoje izvršavanje može se koristiti naredba **break**
- Tada se napušta telo petlje na mestu na kome je izvršena **break** komanda i nastavlja izvršavanje koda neposredno nakon tela petlje
- Koristi se za prisilan izlazak iz tela petlje zbog ispunjenja eksternog uslova

```
for i in range(1,10):  
    if i == 3:  
        break  
    print (i)
```

1
2

```
for broj in range(1,10):  
    if broj == 3:  
        continue  
    print (broj)
```

- Ako postoji potreba da petlja prekine izvršavanje trenutne iteracije i pređe na sledeću može se koristiti naredba **continue**
- Preskaču se sve naredbe do kraja bloka u okviru koga je pozvana **continue** naredba i nastavlja izvršavanje naredne iteracije petlje
- Naredba **continue** se koristi u slučajevima kada u pojedinim iteracijama nije potrebno izvršiti sve naredbe petlje (npr. za parne ili neparne brojeve)

FOR petlja - primeri

Primer for petlje:

```
for letter in 'Python':      # slovo po slovo
    print ('Current Letter :', letter)
fruits = ['banana', 'apple', 'mango']
for fruit in fruits:         # clan po clan
    print ('Current fruit :', fruit)
```

Primer for petlje po indeksu:

```
fruits = ['banana', 'apple', 'mango']
for index in range(len(fruits)):
    print ('Current fruit :', fruits[index])
```

Primer kombinacije for petlje i else:

```
for num in range(10,20):      # od 10 do 19
    for i in range(2,num):    # trazi se delitelj
        if num%i == 0:       # prvi faktor
            j=num/i           # drugi factor
            print ('%d equals %d * %d' % (num,i,j))
            break # probaj sledeci broj
    else:                     # else deo druge for petlje
        print (num, 'is a prime number')
```

FOR petlja - primeri

Gnežđenje petlji:

```
i = 2
while(i < 100):
    j = 2
    while(j <= (i/j)):
        if not(i%j): break
        j = j + 1
    else : print (i, " is prime")
    i = i + 1
```

Prekid petlje pomoću break:

```
for letter in 'Python':      # break u for petlji
    if letter == 'h':
        break
    print ('Current Letter :', letter)
var = 10                      # break u while petlji
while var > 0:
    print ('Current variable value :', var)
    var -= 1

    if var == 5: break
```

FOR petlja - primeri

Korišćenje continue za prelazak na sledeću iteraciju:

```
for letter in 'Python':      # continue u for petlji
    if letter == 'h':
        continue
    print ('Current Letter :', letter)
var = 10                      # continue u while petlji
while var > 0:
    if var == 5:
        continue
    print ('Current variable value :', var)
    var = var -1
```

2. Python funkcije

Funkcije

- Glavni cilj funkcija je modularizacija programskog kôda
- Odgovara razvoju programa u koracima preciziranja
- Funkcije i procedure omogućavaju realizaciju programiranja "odozgo na dole,, što podrazumeva razbijanje problema na potprobleme
- Složeniji programi se mogu podeliti na odgovarajuće logičke i funkcionalne celine, takozvane **potprograme**
- Potprogram omogućava organizovanje određenog broja instrukcija u celine koje obavljaju određene zadatak
- Potprogram preuzima ulazne veličine i vrši njihovu obradu u cilju dobijanja određenih rezultata
- Potprogram možemo pozivati više puta u toku izvršavanja programa
- Rešenje postaje jednostavnije za sagledavanje
- Sistem se lakše održava
- Podržani principi: apstrakcija i dekompozicija

Funkcije

- Funkcija je samostalan deo programa koji obavlja određeni zadatak
- Preko svog naziva i liste parametara, delu programa iz koga je pozvana vraća odgovarajuće rezultate
- Svaka funkcija ima jedinstveni naziv preko koga se može pozvati proizvoljan broj puta iz bilo kog dela programa u cilju izvršenja tog zadatka
- Funkcija mora biti definisana pre prvog korišćenja (poziva)
- Funkcija može biti definisana sa jednim ili više parametara
- Poziv funkcije mora odgovarati definiciji funkcije po broju i redosledu parametara
- Parametri navedeni u opisu funkcije nazivaju se **formalni parametri**
- Parametri navedeni pri pozivu funkcije nazivaju se **stvarni parametri**
- Formalni i stvarni parametri se moraju podudarati po broju i po tipu
- Pri izvršavanju funkcije, formalnim parametrima se dodeljuju vrednosti stvarnih parametara

Definisanje funkcije

```
def ime_funkcije (parametri_funkcije):  
    """docstring funkcije"""  
    telo_funkcije  
    return [izraz]
```

- Funkcija je blok čiji početak označava ključna reč **def** iza koje sledi **ime_funkcije** i zagrade **()**
- Ukoliko postoje parametri funkcije, oni se navode između zagrada
- Opciono se navodi dokumentacioni string funkcije **docstring**
- Blok koji sadrži naredbe funkcije počinje znakom **:** i uvučen je
- Funkcija sadrži naredbu povratka iz funkcije **return**
- Uz naredbu **return** se opciono navodi izraz, čiji rezultat se vraća pozivajućem (pot)programu
- Naredba **return** uz koju nije naveden izraz ekvivalentna je sa **return None**
- **None** je vrednost koja označava odsustvo neke konkretne vrednosti

Funkcije sa parametrima

In Python, parameters are the variables listed in the function definition, while an argument is the value sent to the function when it is called. This distinction is crucial for understanding how functions operate.

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>> def stampaj_2_puta(param):
...     print(param)
...     print(param)
...
>>> stampaj_2_puta('Spam')
Spam
Spam
>>> stampaj_2_puta(math.pi)
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    stampaj_2_puta(math.pi)
NameError: name 'math' is not defined. Did you forget to import 'math'?
>>> import math
>>> stampaj_2_puta(math.pi)
3.141592653589793
3.141592653589793
>>> stampaj_2_puta('Spam'*5)
SpamSpamSpamSpamSpam
SpamSpamSpamSpamSpam
>>> stampaj_2_puta(2025)
2025
2025
>>>
```

Funkcija sa jednim
parametrom

```
>>> def saberi(a,b):
...     c=a+b
...     print(c)
...
>>> saberi(3,5)
8
>>> print(c)
Traceback (most recent call last):
  File "<pyshell#15>", line 1, in <module>
    print(c)
NameError: name 'c' is not defined
>>>
```

Promenljive definisane i
korišćene unutar funkcije,
nisu vidljive van nje

Funkcije sa parametrima - primeri

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>> def srednjaVrednost (lista):
...     """Funkcija nalazi srednju vrednost elemenata liste"""
...     zbir=0
...     for x in lista:
...         zbir+=x
...     return zbir/len(lista)
...
>>> srednjaVrednost([2,5,7,4,12])
6.0
>>> |
```

Funkcija za
izračunavanje srednje
vrednosti elemenata
liste

```
>>> def zbir (*brojevi):
...     rezultat=0
...     for i in brojevi:
...         rezultat+=i
...     return rezultat
...
>>> zbir (27,32, 15, 44, 76, 82, 6, 43)
325
>>> |
```

Broj ulaznih parametara može biti
promenljiv, što se obeležava sa *
ispred naziva ulaznog parametra.
Funkcija izračunava zbir svih brojeva
koji su zadati
kao vrednosti parametara.

Funkcije sa parametrima - primeri

- Definisanjem korisničke funkcije izbegava se ponavljanje sličnih blokova naredbi i program postaje jasniji i kraći

```
zbir = 0
for i in range(1,11):
    zbir = zbir + i
print("Zbir brojeva od 1 do 10 je", zbir)
...
zbir = 0
for i in range(21,61):
    zbir = zbir + i
print("Zbir brojeva od 21 do 60 je", zbir)
...
zbir = 0
for i in range(91,151):
    zbir = zbir + i
print("Zbir brojeva od 91 do 150 je", zbir)
...
```



```
def zbir(n1, n2):
    n = 0
    for i in range(n1, n2+1):
        n = n + i
    return n
...
print("Zbir brojeva od 1 do 11 je", zbir(1,10))
...
print("Zbir brojeva od 21 do 60 je", zbir(21,60))
...
print("Zbir brojeva od 90 do 150 je", zbir(180,220))
...
```

```
>>> def saberi(a,b):
...     c=a+b
...     return c
...
>>> a=saber(12,26)
>>> print(a)
38
```

Promenljiva može
direktno da koristi
rezultat funkcije

Prosleđivanje informacija u funkciju

- Osnovni koncept funkcija jeste prenos argumenata ili prosleđivanje informacija u funkciju:
 - a) Prosleđivanje po vrednosti
 - Stvarni argument je izraz koji se izračunava i njegova vrednost dodeljuje formalnom argumentu
 - Promena vrednosti formalnog argumenta unutar funkcije nema uticaja na stvarni argument
 - b) Prosleđivanje po referenci (adresi)
 - U funkciju se prosleđuje referenca (adresa) stvarnog argumenta
 - Promenom vrednosti formalnog argumenta unutar funkcije menja se stvarni argument
 - c) Prosleđivanje po imenu
 - Efekat kao da se u telu funkcije formalni argument zamenjuje stvarnim argumentom
 - Argument se evaluira pri korišćenju a ne pri pozivu

Prenos po referenci omogućava funkciji da pristupi promenljivoj i promeni njenu vrednost, dok prenos po vrednosti funkciji prenosi samo kopiju trenutne vrednosti promenljive koja je smeštena u sistemski stek poziva

Prosleđivanje informacija u funkciju

- U Pythonu se svi argumenti prenose po referenci tj. koristi se zapravo prenos po referenci objekta
- U Pythonu je svaka promenljiva objekat i objekti mogu biti nepromenljivog tipa *immutable* (bool, int, float, str, tuple) ili promenljivog tipa *mutable* (list, set, dict)
- Kada se objekat nepromenljivog tipa prenosi kao argument, to je slično kao prenos po vrednosti
 - Promena formalnog argumenta unutar funkcije ne utiče na stvarni argument
 - Prilikom te promene zapravo se formira novi objekat, koji ima novi identitet

```
# definicija funkcije
def printme( str ):
    "This prints a passed string into this function"
    print (str);
    return;

# pozivi funkcije
printme("I'm first call to user defined function!");
printme("Again second call to the same function");
    I'm first call to user defined function!
    Again second call to the same function
```

Prosleđivanje informacija u funkciju

- U Pythonu se svi argumenti prenose po referenci

```
# definicija funkcije
def changeme( mylist ):
    "This changes a passed list into this function"
    mylist.append([1,2,3,4]);
    print ("Values inside the function: ", mylist)
    return

# poziv funkcije
mylist = [10,20,30];
changeme( mylist );
print ("Values outside the function: ", mylist)
    Values inside the function: [10, 20, 30, [1, 2, 3, 4]]
    Values outside the function: [10, 20, 30, [1, 2, 3, 4]]
```

Paziti šta znači dodela

```
# definicija funkcije
def changeme( mylist ):
    "This changes a passed list into this function"
    mylist = [1,2,3,4]; # dodela za novu referencu
    print ("Values inside the function: ", mylist)
    return

# poziv funkcije
mylist = [10,20,30];
changeme( mylist );
print ("Values outside the function: ", mylist)
    Values inside the function: [1, 2, 3, 4]
    Values outside the function: [10, 20, 30]
```

Argument je prosleđen preko reference a zatim je referenca prepisana unutar pozvane funkcije

Funkcija bez parametara

```
def pozdravi():  
    """Ova funkcija ispisuje jednostavnu poruku pozdrava."""  
    print("Zdravo!")  
  
# Pozivanje funkcije  
pozdravi()
```

- `def pozdravi():` Ovo definiše funkciju pod imenom `pozdravi`; zagrade `()` označavaju da funkcija ne prima nikakve parametre
- `"""Ova funkcija ispisuje jednostavnu poruku pozdrava."""`: Ovo je docstring (dokumentacioni string), preporuka je da funkcija ima docstring koji objašnjava šta funkcija radi i može mu se pristupiti pomoću `pozdravi.__doc__` (dve donje crte)
- `print("Zdravo!")` Ovo je telo funkcije. U ovom slučaju ispisuje se tekst "Zdravo!".
- `pozdravi()` Vrší se poziv funkcije

Funkcija sa jednim parametrom

```
def pozdravi_osobu(ime):  
    """Ova funkcija pozdravlja osobu sa datim imenom."""  
    print(f"Zdravo, {ime}!")  
  
# Pozivanje funkcije sa argumentom  
pozdravi_osobu("Ana")  
pozdravi_osobu("Marko")
```

- `def pozdravi_osobu(ime):` Funkcija `pozdravi_osobu` prima jedan parametar, koji se zove `ime`
- `print(f"Zdravo, {ime}!")` Unutar funkcije, vrednost parametra `ime` se koristi unutar f-stringa za kreiranje personalizovane poruke

Funkcija sa više parametara

```
def saberi(a, b):  
    """Ova funkcija sabira dva broja i vraća njihov zbir."""  
    zbir = a + b  
    return zbir  
  
# Pozivanje funkcije i čuvanje rezultata  
rezultat = saberi(5, 3)  
print(f"Zbir je: {rezultat}") # Izlaz: Zbir je: 8
```

- `def saberi(a, b):` Funkcija `saberi` prima dva parametra `a` i `b`
- `zbir = a + b` Unutar funkcije parametri se sabiraju i rezultat se čuva u promenljivoj `zbir`
- `return zbir` Ključna reč `return` se koristi za vraćanje vrednosti iz funkcije i u ovom slučaju funkcija `saberi` vraća izračunati zbir brojeva `a` i `b`
- Kada se pozove funkcija `saberi(5, 3)` vrednost 5 se dodeljuje `a`, a 3 se dodeljuje `b`, funkcija izračunava zbir (8) i vraća tu vrednost koja se zatim dodeljuje promenljivoj `rezultat`

Funkcija sa default vrednostima parametara

```
def predstavi_se(ime, godine=30):  
    """Ova funkcija predstavlja osobu sa datim imenom i godinama (podrazumevano 30)."""  
    print(f"Moje ime je {ime} i imam {godine} godina.")  
  
# Pozivanje funkcije sa oba argumenta  
predstavi_se("Jelena", 25) # Izlaz: Moje ime je Jelena i imam 25 godina.  
  
# Pozivanje funkcije samo sa obaveznim argumentom (ime)  
predstavi_se("Petar") # Izlaz: Moje ime je Petar i imam 30 godina.
```

- `def predstavi_se(ime, godine=30):` Parametar `godine` ima podrazumevanu vrednost od 30. Ako se prilikom pozivanja funkcije ne prosledi vrednost za godine, koristiće se podrazumevana vrednost. Parametri sa podrazumevanim vrednostima moraju doći posle parametara bez podrazumevanih vrednosti u definiciji funkcije

Funkcija koja vraća više vrednosti

```
def podeli_i_ostatak(brojilac, imenilac):  
    """Ova funkcija deli dva broja i vraća količnik i ostatak."""  
    kolicnik = brojilac // imenilac  
    ostatak = brojilac % imenilac  
    return kolicnik, ostatak  
  
# Pozivanje funkcije i raspakivanje vraćenih vrednosti  
rez_kolicnik, rez_ostatak = podeli_i_ostatak(17, 5)  
print(f"Količnik: {rez_kolicnik}, Ostatak: {rez_ostatak}") # Izlaz: Količnik: 3, Ostatak: 2
```

- Funkcija `podeli_i_ostatak` koristi operator celobrojnog deljenja (`//`) i operator modula (`%`) da izračuna količnik i ostatak
- Ključna reč `return` vraća više vrednosti odvojene zarezima i Python ovo tretira kao jednu tuple (neizmenljivu sekvencu)
- Prilikom pozivanja funkcije, vraćena tuple se može "raspakovati" u više promenljivih (u ovom slučaju `rez_kolicnik` i `rez_ostatak`)

Funkcija sa proizvoljnim brojem pozicionih parametara (*args)

```
def saberi_sve(*brojevi):  
    """Ova funkcija sabira proizvoljan broj brojeva."""  
    suma = 0  
    for broj in brojevi:  
        suma += broj  
    return suma  
  
# Pozivanje funkcije sa različitim brojem argumenata  
print(saberisve(1, 2, 3))      # Izlaz: 6  
print(saberisve(10, 20, 30, 40)) # Izlaz: 100  
print(saberisve())             # Izlaz: 0
```

- (*brojevi) Sintaksa *args u definiciji funkcije `saberisve` omogućava da funkcija primi proizvoljan broj pozicionih parametara. Ovi parametri se unutar funkcije sakupljaju u strukturu tuple pod imenom `brojevi`

Funkcija sa proizvoljnim brojem ključnih reči parametara (**kwargs)

```
def ispisi_detalje(**podaci):  
    """Ova funkcija ispisuje detalje prosledene kao ključne reči."""  
    for kljuc, vrednost in podaci.items():  
        print(f"{kljuc}: {vrednost}")  
  
# Pozivanje funkcije sa ključnim rečima argumentima  
ispisi_detalje(ime="Marija", grad="Beograd", godine=28)  
# Izlaz:  
# ime: Marija  
# grad: Beograd  
# godine: 28
```

- (**podaci) Sintaksa **kwargs omogućava da funkcija primi proizvoljan broj parametara sa ključnim rečima i ovi parametri se unutar funkcije sakupljaju u rečnik (dictionary) pod imenom podaci, gde su ključne reči ključevi rečnika, a prosledene vrednosti su vrednosti rečnika

Funkcije: Primeri

1) Površina kruga se računa kao: $area = \pi \times r \times r$. Napisati funkciju koja računa area_of_circle:

```
def area_of_circle(r):  
    return 3.14*r**2
```

2) Napisati funkciju add_all_nums kojom se sabiraju argumenti liste. Proveriti su svi elementi liste tipa broj, a ako nisu, treba vratiti povratnu informaciju

```
def add_all_nums(*nums):  
    for i in nums:  
        if type(i) != int and type(i) != float:  
            return f"ERROR: All arguments must be numbers! {i}: {type(i)}"  
    return sum(nums)
```

3) Napisati funkciju sum_two_numbers koja koristi 2 parametra argumenta za sabiranje brojeva:

```
def sum_two_numbers (num_one, num_two):  
    sum = num_one + num_two  
    return sum  
  
print('Sum of two numbers: ', sum_two_numbers(1, 9))
```

Anonimne (lambda) funkcije

- Naziv su dobile jer se ne deklariraju standardno sa `def`, već se koristi ključna reč `lambda`
- Lambda funkcija je tipično mala anonimna (bezimena) funkcija
- Može imati proizvoljan broj argumenata
- Sadrži samo jedan izraz čiji rezultat vraća
- Nema naredbe `return`
- Ne sadrži komande ili višestruke izraze
- Lambda funkcije koriste sopstveni prostor imena
- Sintaksa lambda funkcije

`lambda` argumenti: `izraz`

Anonimne (lambda) funkcije

```
>>> (lambda x,y:x+y) (2,3)  #bezimena lambda f-ja
5
>>> x=lambda a,b: a*b      #Dodeljeno ime x lambda f-ji
>>> print(x(5,7))
35
>>> |
```

Bezimena i imenovana
lambda funkcija

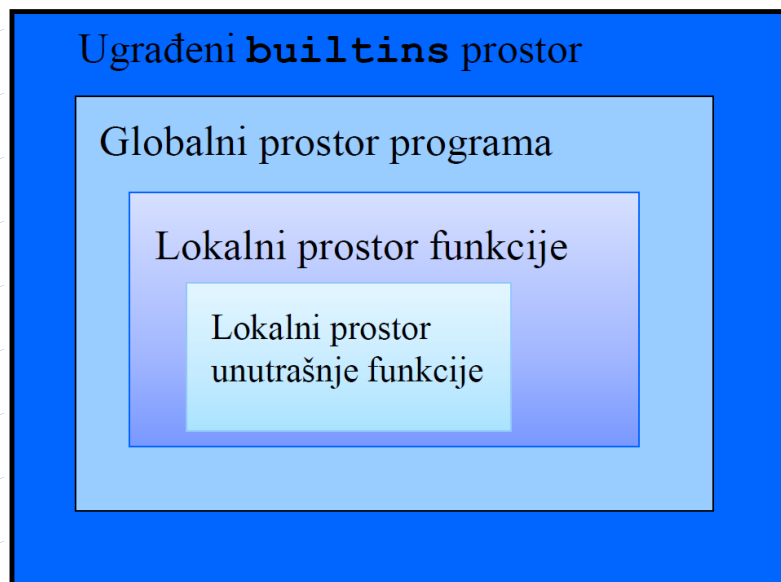
```
>>> def mnozac(n):
...     return lambda a: a*n
...
>>> dvostruko=mnozac(2)
>>> trostruko=mnozac(3)
>>> print(dvostruko(55))
110
>>> print(trostruko(111))
333
>>> |
```

Kombinovanje lambda i
klasične funkcije

```
sum = lambda arg1, arg2: arg1 + arg2;
print ("Value of total : ", sum( 10, 20 ))
print ("Value of total : ", sum( 20, 20 ))
Value of total : 30
Value of total : 40
```

Opseg važenja promenljivih

- Promenljive predstavljaju imena za objekte i veza imena i objekta se uspostavlja dodelom vrednosti
- Prostor imena (*namespace*) je preslikavanje između naziva objekata i samih objekata (promenljivih i funkcija)
- Prostor imena realizuje se interno tabelom imena
- Tokom izvršavanja programa postoje bar dva (osnovna) prostora imena
- Isto ime može postojati u više prostora imena i tada se odnosi na različite objekte



Prostor imena je
hijerarhijski
organizovan

Opseg važenja promenljivih

- Dostupnost ili oblast definisanosti (*scope*) promenljive predstavlja deo programa u kojem je promenljiva definisana i može se koristiti
- Lokalne promenljive nastaju i dostupne su samo u okviru funkcije u kojoj se koriste; Nakon završetka funkcije one više nisu dostupne a nakon svakog narednog izvršavanja funkcije one se ponovo kreiraju
- Globalne promenljive definisane u glavnom programu imaju globalnu dostupnost i mogu se koristiti iz svake funkcije
- Globalnu dostupnost imaju i ugrađeni **builtin** objekti
- Ime se najpre traži u lokalnom prostoru imena, a ako se ne pronade, ide se nivo više u hijerarhiji
- Po pokretanju interpretera, formira se prostor imena ugrađenog builtins prostora koji sadrži predefinisana imena npr. **print**
- Kada se program pokrene, kreira se globalni prostor programa
- Python dozvoljava da se funkcija definiše unutar druge funkcije
- Pri pozivu svake funkcije kreira se lokalni prostor imena te funkcije
- Svaki od ovih prostora ima svoj životni vek, koji traje tokom izvršavanja odgovarajućeg koda

Opseg važenja promenljivih

```
IDLE Shell 3.13.3
File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>> def f1():
...     a=2          #Lokalna promenljiva za f1
...     def f2():
...         a=3      #Lokalna promenljiva za f2
...         print(a)
...     f2()
...     print(a)
...
>>> a=1              #Globalna promenljiva
>>> f1(1)
Traceback (most recent call last):
  File "<pyshell#9>", line 1, in <module>
    f1(1)
TypeError: f1() takes 0 positional arguments but 1 was given
>>> f1()
3
2
>>> print(a)
1
>>>
```

Različite vrednosti
promenljivih za lokalne
i globalni kontekst

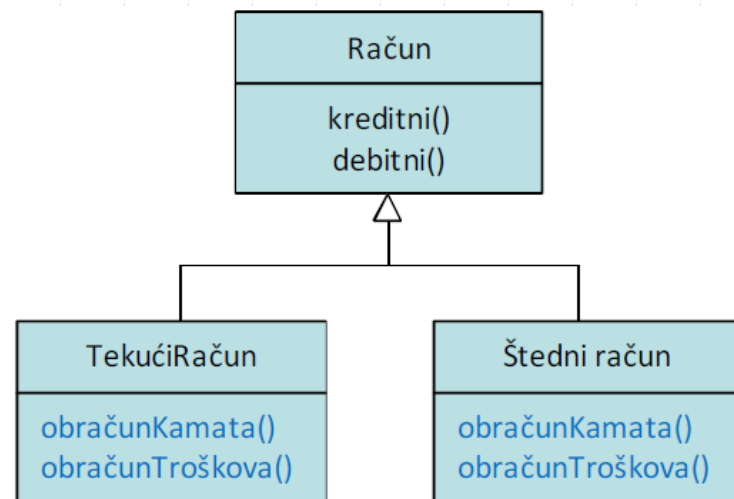
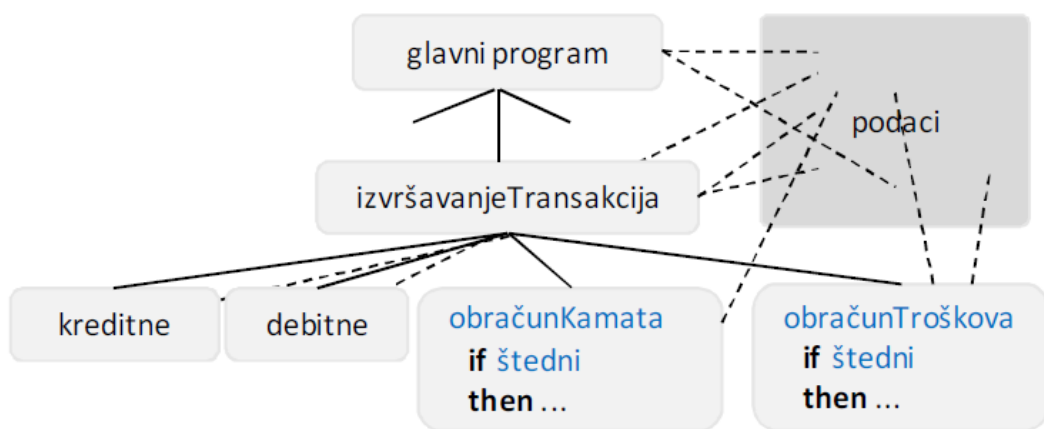
```
>>> def f1():
...     global a
...     a=2          #Globalna promenljiva za f1
...     def f2():
...         global a
...         a=3      #Globalna promenljiva za f2
...         print(a)
...     f2()
...     print(a)
...
>>> f1()
3
3
>>> print(a)
3
>>>
```

Dodavanjem naredbe **global** za
promenljive u *f1* i *f2* ne kreiraju
se lokalne promenljive već se
koristi postojeća globalna

3. Python objektno-orijentisano programiranje (klase)

Objektno-orijentisani razvoj softvera

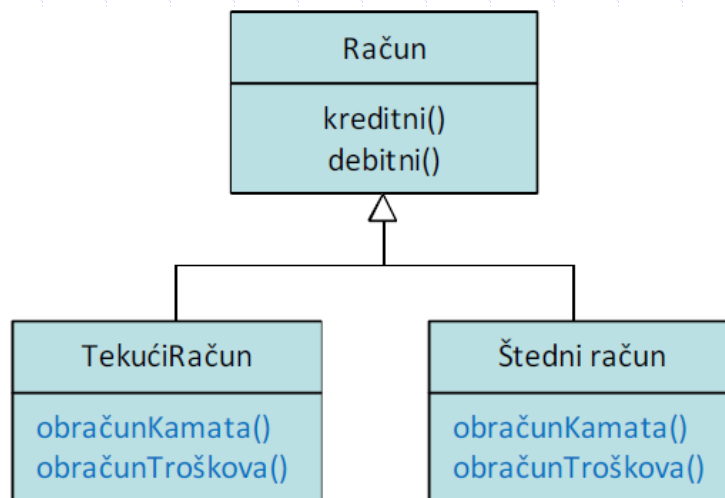
- Objektno-orijentisani (OO) razvoj softvera koristi apstrakciju kao metod smanjenja kompleksnosti procesa razvoja softvera
- Proceduralna apstrakcija, koja se koristi u klasičnom proceduralnom programiranju, organizuje softver prema funkcijama, a apstrakcija podataka se vrši odvojeno, pomoću struktura podataka



Proceduralna i OO organizacija softverskog sistema

Objektno-orijentisani razvoj softvera

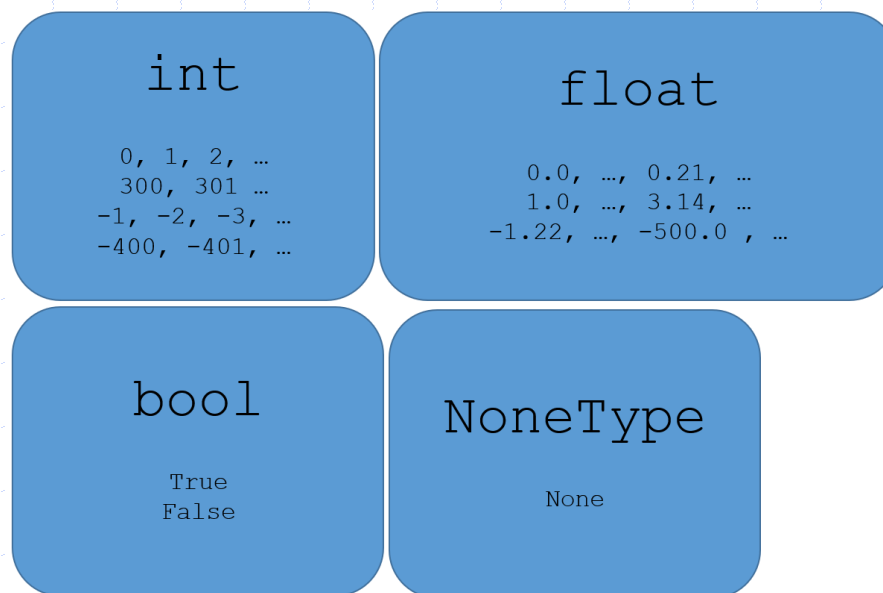
- Objektno-orijentisani pristup posmatra sistem kao skup objekata, koji se sastoje od procedura i podataka, na različitim nivoima apstrakcije i međusobno sarađuju radi izvršenja određenog zadatka
- Kod proceduralnog razvoja podaci i operacije su međusobno razdvojeni, tako da je uvek neophodno slanje podataka metodama
- OO programiranje smešta podatke i operacije koje se odnose na njih zajedno u objektu.



OO organizacija softverskog sistema

Objekti i klase

- U jeziku Python svi podaci, numerički i nenumerički, predstavljaju objekte
- Objektu se prilikom njegovog kreiranja automatski dodeljuje jednoznačna celobrojna identifikacija. Objekti iste vrste su istog tipa

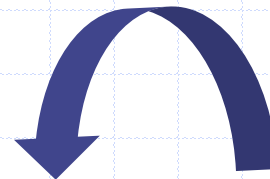


```
>>> n = 3      # n je definisan kao celi broj
>>> id(n)      # identifikacija dodeljena promenljivoj n
1801060880
>>>> type(n) # tip promenljive n
<class 'int'>
```

Objekti i metode

- U jeziku Python tip objekta definisan je klasom kojoj pripada (npr. klasa celobr. tipa je 'int', decim. broja je 'float', a tipa string je 'str')
- Termin **tip** (*type*) i **klasa** (*class*) imaju isto značenje, a sama promenljiva je referenca na objekt
- Operacije nad objektima definisane su funkcijama koje se nazivaju **metode**
- Metode mogu pokrenuti određeni objekti, npr. tip string ima metode kao što su `lower()` i `upper()`, koji kao rezultat vraćaju novi string sa malim ili velikim slovima:

```
>>> s = "Dobrodošli"  
>>> s.lower()  
' dobrodošli'
```



Sintaksa za pokretanje
metode objekta

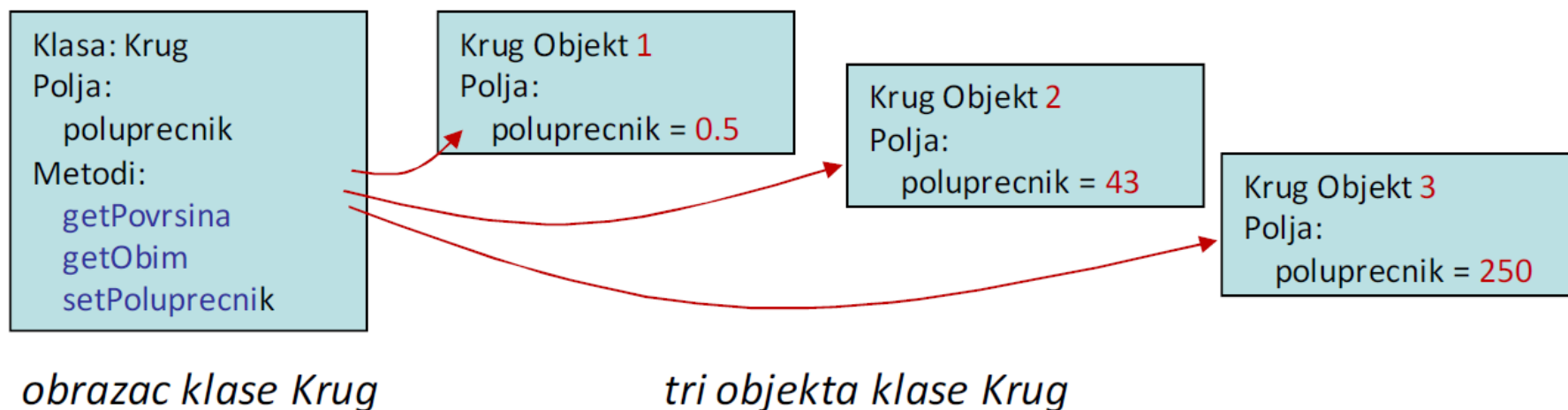
```
<objekt>.<metod>()
```

Objekti i metode

- Objekat ima jedinstven identitet, stanje i ponašanje:
 - identitet je jedinstveni broj koji se dodeljuje svakom objektu
 - stanje se prikazuje pomoću promenljivih ili polja atributa (svojstava), kao npr. poluprečnik kruga, visina i širina pravougaonika itd.
 - ponašanje su metode ili funkcije određenog objekta, koje se pokreću radi realizacije neke akcije vezane za objekat
- Apstrakcija klase je razdvajanje implementacije same klase od njene upotrebe; Korisnik klase zna samo da je koristi, pri čemu opisi metoda i njihovog ponašanja služe kao ugovor (*contract*) između klase i klijenta
- Enkapsulacija je skrivanje od korisnika detalja načina objedinjavanja podataka i metoda u jednu celinu; U stvarnosti su detalji implementacije različitih pojava, odnosno sistema, po pravilu skriveni od korisnika (npr. tehnički ili računarski sistemi za koje korisnici znaju samo funkcije i način njihove upotrebe...)

Kreiranje objekata

- **Klasa** (*class*) predstavlja opis objekata iste vrste
- U Pythonu korisnički definisana klasa koristi promenljive kao polja za smeštanje podataka i definiše metode za izvršavanje akcija
- Klasa predstavlja obrazac (*template*), nacrt (*blueprint*) ili ugovor (*contract*)
- Objekat je **primerak** ili **instanca** (*instance*) klase, a kreiranje instance klase se naziva instancijacija (*instantiation*)



Instance klase

- Pretpostavimo da treba da kreiramo instancu koordinate

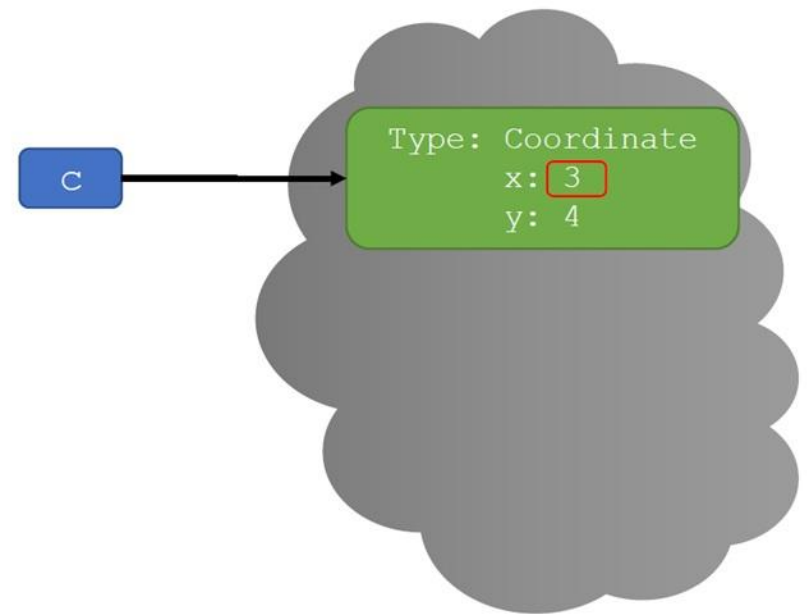
```
c = Coordinate(3,4)
```

- Instancu formiramo kao memorijsku strukturu

- Izvršavanjem funkcije

```
c.x
```

pretražujemo strukturu na koju `c` ukazuje, a zatim pronalazimo vezu sa `x` u toj strukturi



Instance klase

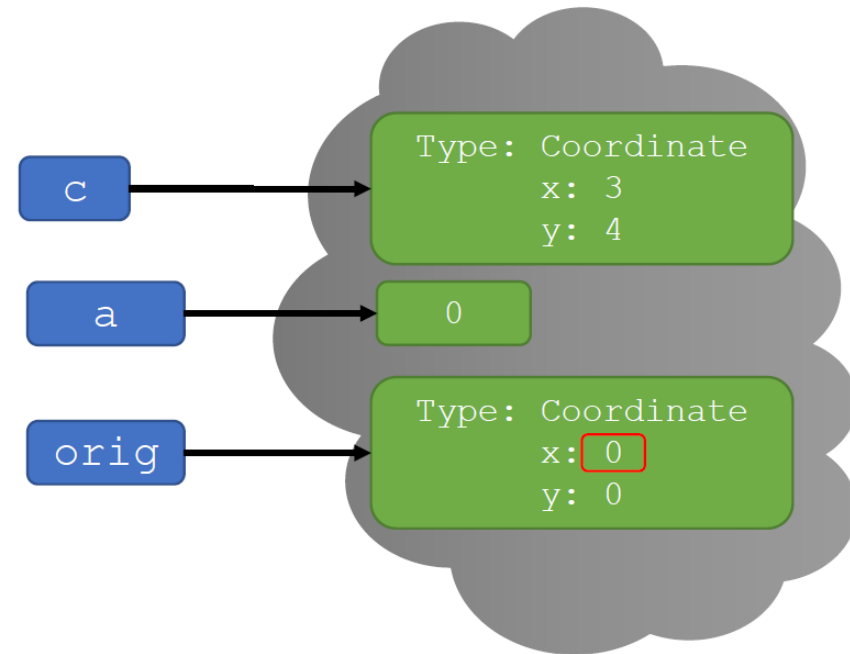
Kreiramo drugu instancu preko promenljive

`a=0`

`orig = Coordinate(a,a)`

`orig.x`

- Sve su ovo objekti u memoriji
- Mi samo pristupamo atributima ovih objekata



Definisanje klase

- Klasa objekata u jeziku Python se definiše naredbom oblika:

```
class <NazivKlase>:  
    <inicijalizator>  
    <metodi klase>
```

- Klasa za smeštanje podataka koristi polja (atribute)
- Prilikom kreiranja novog objekta klase, za inicijalizaciju podataka koristi se **inicijalizator**, poseban metod rezervisanog imena `__init__()`
- Inicijalizator ima najmanje jedan parametar (`self`), ali se u parametar listi mogu deklarirati dodatni parametri za prenos različitih vrednosti inicijalizatoru
- Ako korisnička klasa nema polja podataka koja treba inicijalizovati, ne mora da ima definiciju sopstvenog inicijalizatora

Definisanje klase

- Python koristi `class` ključnu reč za definisanje novog tipa

class definition *name/type* *class parent*

```
class Coordinate(object):  
    #define attributes here
```

- Slično kao kod funkcija i ključne reči `def`, koristi se uvlačenje koda kako bi se označilo koji iskazi pripadaju definiciji klase
- Reč `object` znači da `Coordinate` predstavlja Python objekat i nasleđuje sve njegove attribute
- Podaci i procedure "pripadaju" klasi
- Metode** (atributi procedura) su funkcije koje se izvršavaju samo sa definisanim klasom

Definisanje klase

- Primer definisanja klase **Krug**:

```
import math
class Krug:
    def __init__(self, poluprecnik = 1):
        self.poluprecnik = poluprecnik
    def getObim(self):
        return 2 * self.poluprecnik * math.pi
    def getPovrsina(self):
        return self.poluprecnik **2 * math.pi
    def setPoluprecnik(self, poluprecnik):
        self.poluprecnik = poluprecnik
```

- Inicijalizator **__init__** kreira novu promen. **poluprecnik** u objektu
- To omogućava sistemska promenljiva **self**, koja u objektu predstavlja referencu na sam taj objekat
- Metode klase su funkcije za uvid u stanje podataka objekta klase **getObim()** i **getPovrsina()** ili "geteri" i funkcije za postavljanje vrednosti polja podataka **setPoluprecnik()** ili „seteri“

Kreiranje objekata konstruktorom

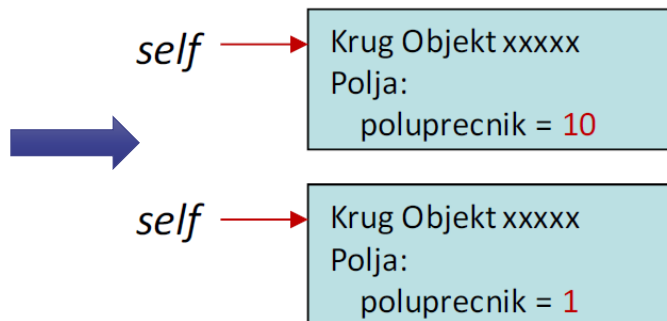
- Nakon definisanja klase, objekti se kreiraju pomoću **konstruktor**a, koji kreira u memoriji objekt određene klase i pokreće metod `__init__` za inicijalizaciju objekta
- Svi metodi unutar objekta koriste kao prvi argument **self**, referencu na sam objekt, čija vrednost se definiše prilikom kreiranja objekta
- Sintaksa pokretanja konstruktora klase je:

```
<NazivKlase>(<argumenti>)
```

- Kreiranje objekata klase Krug:

```
>>> a = Krug(10)
>>> b = Krug()           # default poluprecnik=1
```

Prvi objekt
je inicijalizovan na zadanu, a drugi na
podrazumevanu vrednost polja



Pristup članovima objekta

- Pristup članovima, podacima i metodima klasa, vrši se korišćenjem naziva objekta i operatora "."(tačka):

```
<promenljivaObj> = <NazivKlase>(<argumenti>)  
<promenljivaObj>.<polje>  
<promenljivaObj>.<metod>(<argumenti>)
```

- Npr. pristup članovima objekt klase **Krug** poluprečnika 20 i postavljanje nove vrednosti poluprečnika kruga:

```
>>> prom_krug = Krug(20)  
>>> prom_krug.poluprecnik  
20  
>>> prom_krug.getObim()  
125.66370614359172  
>>> prom_krug.getPovrsina()  
1256.6370614359173  
>>> prom_krug.setPoluprecnik(5)  
>>> prom_krug.poluprecnik  
5
```

Parametar `self`

- Parametar `self` omogućava referisanje nekog objekta na samog sebe, odnosno omogućava metodama klase da u konkretnim objektima pristupaju članovima klase koristeći izraze `self.<nazivPolja>` i `self.<nazivMetode>()`
- Oblast definisanosti promenljivih kreiranih kao `self.<nazivPolja>` je cela klasa
- Metodi klase mogu kreirati i lokalne promenljive:

```
class NazivKlase:
    def __init__(self, ...):
        self.x = 1      # promenljiva x na nivou klase
    ...
    def metod1(self, ...):
        self.y = 2      # promenljiva y na nivou klase
    ...
    z = 5 # lokalna promenljiva na nivou metoda1
```

Upotreba kreirane klase

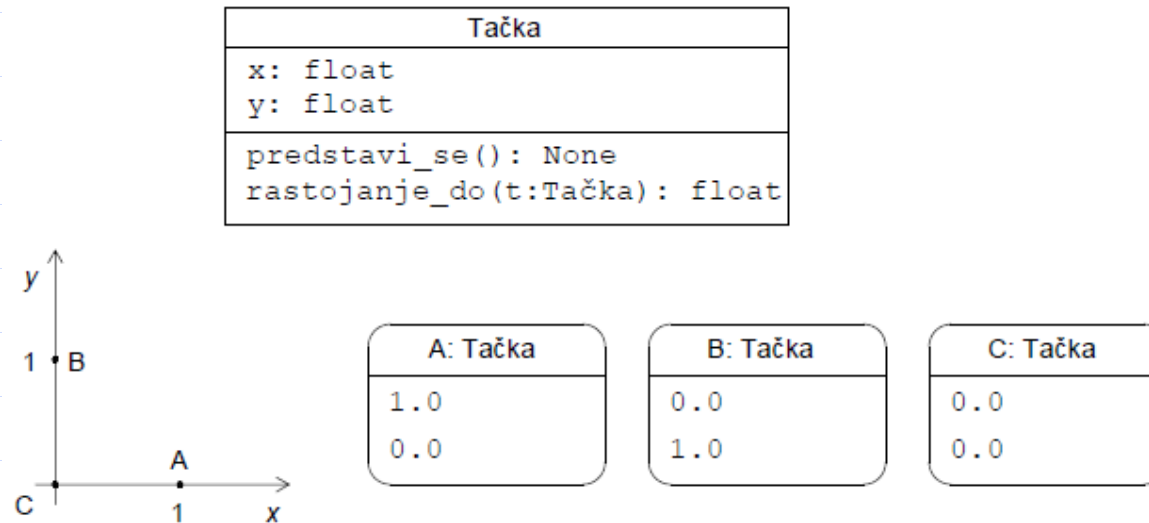
```
from krug import Krug

# Kreiranje objekta tipa krug s poluprečnikom 1
krug1 = Krug()
print("Površina kruga poluprečnika", \
      krug1.poluprecnik, "je", krug1.getPovrsina())

# Kreiranje objekta tipa krug s poluprečnikom 250
krug2 = Krug(250)
print("Površina kruga poluprečnika", \
      krug2.poluprecnik, "je", krug2.getPovrsina())
# Promena poluprečnika objekta krug2 u 100
krug2.poluprecnik = 100      # ili setPoluprecnik(100)
print("Površina kruga poluprečnika", \
      krug21.poluprecnik, "je", krug2.getPovrsina())
```

```
Površina kruga poluprečnika 1 je 3.141592653589793
Površina kruga poluprečnika 250 je 196349.54084936206
Površina kruga poluprečnika 100 je 31415.926535897932
>>>
```

Primer



- Klasa `Tačka` modeluje tačke u ravni, sve tačke opisane su sa dva atributa `x` i `y` koja predstavljaju koordinate u ravni
- Metodama `predstavi_se()` i `rastojanje_do()` ispisuje se par koordinata o konkretnoj tački (objektu) tj. vraća se realni broj koji se odnosi na rastojanje od tačke (objekta) nad kojom se poziva, do tačke koja se prosleđuje metodi kao parametar
- U prikazu klase, pored atributa navedeni su njihovi tipovi, a pored metoda tipovi povratnih vrednosti i tipovi ulaznih parametara

Primer

```
1 from math import sqrt
2 # definiše apstraktni tip koji predstavlja sve tačke u ravni
3 class Tačka:
4     ''' Predstavlja tačke u ravni definisane sa x i y koord'''
5
6     # poziva se iz konstruktora pri kreiranju objekta
7     def __init__(self, x = 0, y = 0):
8
9         self.x = x      # definisanje atributa x
10        self.y = y      # definisanje atributa y
11
12        # ispisuje podatke o objektu
13        def predstavi_se(self):
14
15            print(f'({self.x:<f}, {self.y:<f})')
16
17        # računa rastojanje do druge tačke t
18        def rastojanje_do(self, t):
19
20            dx = self.x - t.x
21            dy = self.y - t.y
22            return sqrt(dx**2 + dy**2)
```

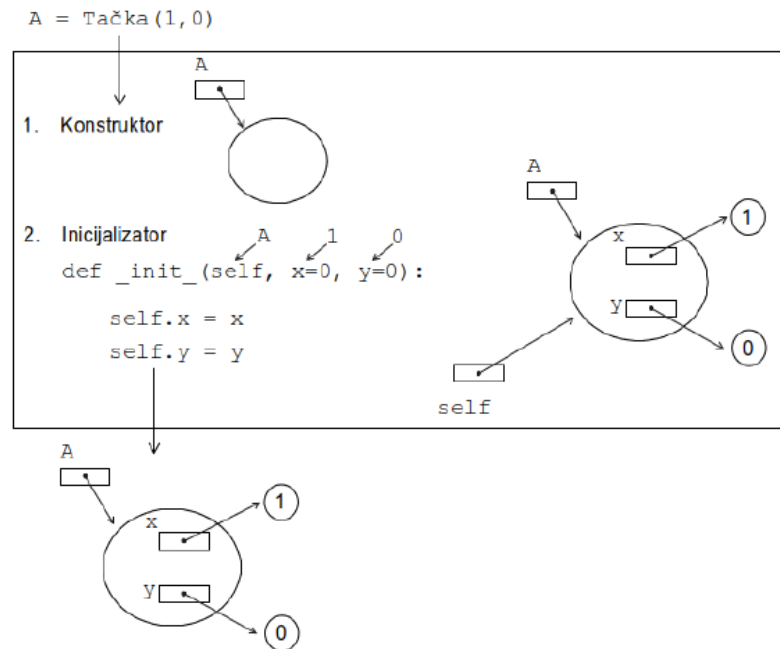
- Klasa Tačka definisana je u modulu `tacke.py`

Primer

```
>>> from tacke import Tačka
>>> A = Tačka(1,0) # poziv konstruktora klase Tačka
>>> type(A)
<class 'tacke.Tačka'>
>>> A.x           # pristupanje atributu x
1
>>> A.y           # pristupanje atributu y
0
>>> C = Tačka()   # poziv konstruktora sa podrazumevanim vrednostima
>>> print(C.x, C.y)
0 0
```

- U bloku klase definišu se dve metode kao i specijalna metoda `__init()` koja se poziva svaki put kada se kreira novi objekat
- Prilikom izvršavanja reda `A = Tacka(1,0)` poziva se konstruktor klase `Tačka` kojim se kreira novi objekat
- Konstruktor ne inicijalizuje stanje objekta, već kreira potrebnu memorijsku strukturu za čuvanje budućeg sadržaja
- Ako se u definiciji klase pojavi metoda `__init()` onda je konstruktor obavezno poziva i prosleđuje joj navedene argumente (linije koda 7-10, u pozivu konstruktora za tačku A argumenti su 1 i 0)

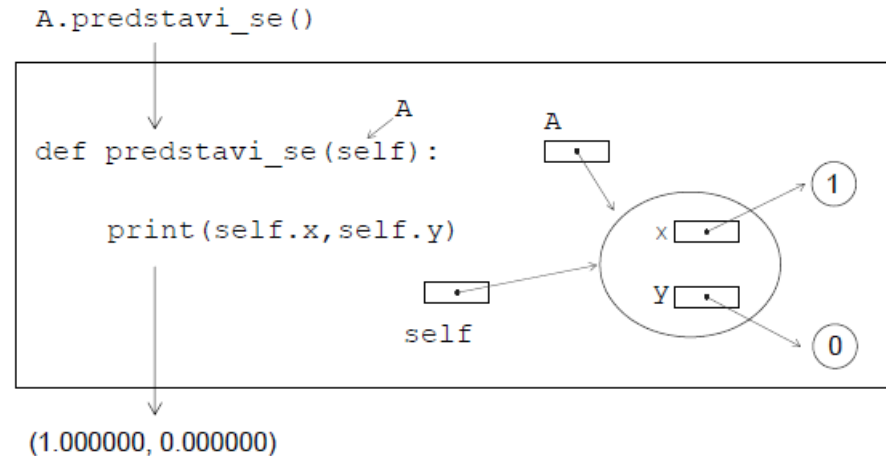
Primer



- Postupak kreiranja novog objekta: konstruktor prvo kreira prazan objekat pa potom, ako je definisana u klasi, poziva metodu __init__()__ koja formira i inicijalizuje attribute
- Prilikom poziva inicijalizatora, referenca A kopira se u referencu self pa obe pokazuju na novokreirani objekat.
- U inicijalizatoru se atributima pristupa u notaciji sa tačkom preko reference self (linije koda 9, 10)

Primer

```
>>> A = Tačka(1,0)
>>> B = Tačka(0,1)
>>> C = Tačka()
>>> A.predstavi_se()
(1.000000, 0.000000)
>>> A.rastojanje_do(C)
1.0
>>> A.rastojanje_do(B)
1.4142135623730951
```



- Metoda `predstavi_se()` ispisuje koordinate tačke nad kojom je pozvana (linija 13-15) i tokom njenog poziva se ne navode argumenti, iako njen potpis sadrži ulazni parametar `self` (linija 13). Prilikom poziva metode instance nad objektom `x`, referenca `x` automatski se kopira u prvi argument metode, u liniji 15 se ispisuju vrednosti koordinata tačke na koju ukazuje `self` (`self.x` i `self.y`). Potpis metode instance uvek sadrži jedan parametar više (`self`), u odnosu na broj argumenata u pozivu
- Metoda `rastojanje_do()` pored reference `self`, prima i parametar `t` koji ukazuje na tačku do koje se računa rastojanje (linija 18). Promenljive `self.x` i `self.y` ukazuju na koordinate tačke nad kojom je metoda pozvana, a `t.x` i `t.y` na koordinate tačke koja je predstavljena parametrom `t` (linije 20, 21). Rastojanje se računa prema formuli za euklidsko rastojanje koje se u ravni svodi na Pitagorinu teoremu (linija 22)

